Autokamak: An Agentic Pipeline for Learning Grad–Shafranov Equilibrium Surrogates

A Mathematical System Description

Hindy Rossignol

July 18, 2026

Abstract

Autokamak is a research platform that couples a finite-element Grad–Shafranov (GS) equilibrium solver (TokaMaker/OpenFUSIONToolkit) with a machine-learning pipeline that learns fast surrogate models for the poloidal flux field $\psi(R, Z)$, and with an LLM-driven *meta-agent* that plans and runs the pipeline end to end. The system is organized in three phases: (i) generation of a seed dataset by space-filling sampling of tokamak geometries; (ii) automated surrogate training with dimensionality reduction and hyperparameter search; and (iii) *residual-driven active sampling*, in which a Gaussian process learns *where the current surrogate is wrong* as a function of geometry and proposes the next equilibria to solve. An outer meta-loop selects among data-enrichment and re-tuning actions until a performance target is met. This document gives a self-contained mathematical account of every model in the pipeline, stating each optimization problem explicitly.

Contents

1 System Overview

We wish to approximate the map from a low-dimensional description of a plasma equilibrium to the full two-dimensional flux field it produces. Let

$$\boldsymbol{x} = (R_0, a, \kappa, \delta, I_p) \in \mathcal{X} \subset \mathbb{R}^5 \tag{1}$$

denote the *geometry vector*: major radius R_0, minor radius a, elongation κ, triangularity δ, and total plasma current I_p. The ground-truth solver returns the poloidal flux $\psi(R, Z)$, which after interpolation onto a fixed $n_z \times n_r$ grid we regard as a vector $\boldsymbol{\psi} \in \mathbb{R}^D$ with $D = n_z n_r$ (in the shipped configuration $n_z = 96$, $n_r = 64$, so $D = 6144$). The object we learn is the operator

$$\mathcal{S} : \ \mathcal{X} \to \mathbb{R}^D, \qquad \boldsymbol{x} \mapsto \boldsymbol{\psi}, \tag{2}$$

called the *surrogate*. The three phases of the pipeline correspond to: building a training set $\{(\boldsymbol{x}_i, \boldsymbol{\psi}_i)\}$ (Section 3); fitting $\mathcal{S}$ (Section 4); and choosing new $\boldsymbol{x}$ to solve so as to improve $\mathcal{S}$ where it is weakest (Section 5). An LLM meta-agent orchestrates the loop (Section 6), and an agentic code-generation layer (URSA) authors open-ended pipeline components (Section 7).

2 The Ground-Truth Problem: Grad–Shafranov Equilibrium

A magnetically confined, axisymmetric plasma in force balance satisfies the *Grad–Shafranov equation*, a nonlinear elliptic PDE for the poloidal flux $\psi(R, Z)$ in the poloidal plane (R, Z):

$$\Delta^* \psi \ = \ -\,\mu_0 R^2\, p'(\psi) \ - \ F(\psi)\, F'(\psi), \tag{3}$$

where the elliptic *Shafranov* operator is

$$\Delta^* \psi \ = \ R\,\frac{\partial}{\partial R}\left(\frac{1}{R}\frac{\partial \psi}{\partial R}\right) + \frac{\partial^2 \psi}{\partial Z^2}. \tag{4}$$

Here μ_0 is the vacuum permeability, $p(\psi)$ the plasma pressure profile, and $F(\psi) = RB_\phi$ the poloidal-current function; primes denote $\mathrm{d}/\mathrm{d}\psi$. The right-hand side is the toroidal current density $-\mu_0 R\, J_\phi$. Equation (3) is solved on the plasma cross-section bounded by the *last closed flux surface* (LCFS) Γ, a closed curve prescribed (in the fixed-boundary setting used here) by the shaping parameters through a Miller-type parameterization

$$R(\theta) = R_0 + a\cos\big(\theta + \arcsin\delta \cdot \sin\theta\big), \qquad Z(\theta) = z_0 + \kappa\, a \sin\theta, \qquad \theta \in [0, 2\pi). \tag{5}$$

TokaMaker discretizes (3) with a finite-element method on an unstructured triangular mesh of the region enclosed by Γ, imposing the total current constraint $\int J_\phi \,\mathrm{d}R\,\mathrm{d}Z = I_p$ and an isoflux boundary condition matching Γ. The nonlinearity in p' and FF' is resolved by Picard iteration.

The solver is the *oracle*: given $\boldsymbol{x}$, it returns ψ on the mesh, which we bilinearly interpolate to the fixed (R, Z) grid to obtain $\boldsymbol{\psi} \in \mathbb{R}^D$. Cells outside Γ are undefined and marked `NaN`. A solve occasionally fails (mesh degeneracy or non-convergence at extreme shaping); we record a *feasibility* indicator $y^{\text{feas}} \in \{0, 1\}$ per attempt, which becomes important in Section 5.

Each solve costs seconds of wall-clock; a surrogate that emulates $\mathcal{S}$ in microseconds is the goal, and the cost asymmetry is what makes *which equilibria to solve* an economically meaningful question.

3 Phase 1: Seed Dataset Generation

3.1 Sampling designs

Phase 1 draws N geometries from a bounded box $\mathcal{B} = \prod_{d=1}^{5}[\ell_d, u_d] \subseteq \mathcal{X}$ (the *seed box*) and solves each. Because we cannot yet know where the eventual surrogate will struggle, the seed draw is *blind*; the design goal is uniform coverage. Three seeded designs are supported.

Uniform. Independent draws $\boldsymbol{x}_i \sim \text{Unif}(\mathcal{B})$. Simple, but at finite N leaves clusters and gaps by chance.

Latin Hypercube Sampling (LHS). Partition each axis into N equal bins and permute so that every bin is occupied exactly once per dimension. With $\pi^{(d)}$ a random permutation of $\{0, \ldots, N-1\}$ and $U_i^{(d)} \sim \text{Unif}(0, 1)$,

$$x_{i,d} = \ell_d + (u_d - \ell_d)\,\frac{\pi_i^{(d)} + U_i^{(d)}}{N}. \tag{6}$$

LHS guarantees one-dimensional uniformity by construction.

Sobol (low-discrepancy). A deterministic, scrambled Sobol sequence minimizes the *star discrepancy*

$$D_N^{\star} \;=\; \sup_{\boldsymbol{b} \in \mathcal{B}} \left| \frac{\#\{\boldsymbol{x}_i \in [\ell, \boldsymbol{b})\}}{N} - \frac{\text{vol}[\ell, \boldsymbol{b})}{\text{vol}\,\mathcal{B}} \right|, \tag{7}$$

which measures the worst-case deviation between the empirical fraction of points in any sub-box and its volume fraction. Low-discrepancy sequences achieve $D_N^{\star} = O\big((\log N)^5/N\big)$ versus $O(N^{-1/2})$ for i.i.d. uniform, giving the most even coverage of $\mathcal{B}$ in five dimensions. The same scrambled-Sobol routine supplies the acquisition candidate pool in Section 5.

3.2 Dataset

Solving the N designs yields the seed dataset

$$\mathcal{D}_N = \big\{(\boldsymbol{x}_i,\, \boldsymbol{\psi}_i,\, y_i^{\text{feas}})\big\}_{i=1}^{N}, \tag{8}$$

where $\boldsymbol{\psi}_i$ is defined only for feasible solves. We store *physical* ψ in webers (not the normalized $\psi_N \in [0, 1]$), because normalization erases the I_p amplitude dependence the surrogate must learn.

4 Phase 2: Surrogate Modeling

Direct regression onto $\mathbb{R}^D$ ($D \approx 6144$) from five inputs is ill-posed. Phase 2 therefore factors $\mathcal{S}$ into a linear dimensionality reduction learned from data and a low-dimensional nonlinear regressor:

$$\mathcal{S}(\boldsymbol{x}) \;=\; \boldsymbol{\mu}_\psi + \boldsymbol{W}\, h(\boldsymbol{x}), \tag{9}$$

with $\boldsymbol{\mu}_\psi \in \mathbb{R}^D$ a mean field, $\boldsymbol{W} \in \mathbb{R}^{D\times k}$ an orthonormal basis, and $h : \mathcal{X} \to \mathbb{R}^k$ the regressor ($k \ll D$).

4.1 Principal Component Analysis of the flux field

Let $\boldsymbol{\mu}_\psi = \frac{1}{n}\sum_i \boldsymbol{\psi}_i$ (per-pixel mean; `NaN` cells are mean-filled). PCA seeks the rank-k orthonormal basis minimizing reconstruction error, equivalently maximizing retained variance:

$$\boldsymbol{W}^\star = \underset{\boldsymbol{W}\in\mathbb{R}^{D\times k},\, \boldsymbol{W}^\top\boldsymbol{W}=\boldsymbol{I}_k}{\arg\min} \sum_{i=1}^{n} \big\| (\boldsymbol{\psi}_i - \boldsymbol{\mu}_\psi) - \boldsymbol{W}\boldsymbol{W}^\top(\boldsymbol{\psi}_i - \boldsymbol{\mu}_\psi) \big\|_2^2. \tag{10}$$

The solution is the matrix whose columns are the top-k eigenvectors of the empirical covariance $\boldsymbol{\Sigma} = \frac{1}{n}\sum_i(\boldsymbol{\psi}_i - \boldsymbol{\mu}_\psi)(\boldsymbol{\psi}_i - \boldsymbol{\mu}_\psi)^\top$, ordered by eigenvalue $\lambda_1 \geq \cdots \geq \lambda_k$. Each field is encoded by its coefficient vector and reconstructed by

$$\boldsymbol{c}_i = \boldsymbol{W}^{\star\top}(\boldsymbol{\psi}_i - \boldsymbol{\mu}_\psi) \in \mathbb{R}^k, \qquad \hat{\boldsymbol{\psi}}_i = \boldsymbol{\mu}_\psi + \boldsymbol{W}^\star \boldsymbol{c}_i. \tag{11}$$

The retained-variance fraction is $\mathrm{EV}(k) = \sum_{j\leq k}\lambda_j / \sum_j \lambda_j$; we choose the smallest k with $\mathrm{EV}(k) \geq 0.95$ (physical ψ typically needs $k \approx 8$). The regression target is thus the compact $\boldsymbol{c} \in \mathbb{R}^k$, not the 6144-dimensional field.

4.2 The regressor zoo

The regressor $h : \mathcal{X} \to \mathbb{R}^k$ is chosen from four classical families. All operate on standardized inputs $\tilde{\boldsymbol{x}} = (\boldsymbol{x} - \bar{\boldsymbol{x}})/\boldsymbol{s}$ (component-wise; $I_p \sim 10^5$ dwarfs $R_0 \sim 10^{-1}$ otherwise). We state each family's training objective; multi-output (k coefficients) is handled per output or jointly as noted.

(a) Polynomial ridge regression. With polynomial feature map $\phi_d(\boldsymbol{x})$ of total degree $\leq d$, each coefficient channel j solves a ridge least-squares problem

$$\boldsymbol{\beta}_j^\star = \underset{\boldsymbol{\beta}\in\mathbb{R}^m}{\arg\min} \sum_{i=1}^{n} \big(c_{i,j} - \boldsymbol{\beta}^\top \phi_d(\tilde{\boldsymbol{x}}_i)\big)^2 + \alpha\, \|\boldsymbol{\beta}\|_2^2, \tag{12}$$

with closed-form solution $\boldsymbol{\beta}_j^\star = (\boldsymbol{\Phi}^\top\boldsymbol{\Phi} + \alpha\boldsymbol{I})^{-1}\boldsymbol{\Phi}^\top \boldsymbol{c}_{:,j}$, where $\boldsymbol{\Phi}$ stacks $\phi_d(\tilde{\boldsymbol{x}}_i)^\top$. Hyperparameters: α (ridge penalty), $d \in \{1, 2, 3\}$.

(b) Kernel ridge regression (KRR). Working in a reproducing-kernel Hilbert space $\mathcal{H}_\kappa$,

$$f_j^\star = \underset{f\in\mathcal{H}_\kappa}{\arg\min} \sum_{i=1}^{n} \big(c_{i,j} - f(\tilde{\boldsymbol{x}}_i)\big)^2 + \lambda\, \|f\|_{\mathcal{H}_\kappa}^2. \tag{13}$$

By the representer theorem the minimizer is $f_j^\star(\boldsymbol{x}) = \sum_i \alpha_i\, \kappa(\boldsymbol{x}, \boldsymbol{x}_i)$ with dual weights

$$\boldsymbol{\alpha}_j = (\boldsymbol{K} + \lambda\boldsymbol{I})^{-1}\, \boldsymbol{c}_{:,j}, \qquad K_{ii'} = \kappa(\tilde{\boldsymbol{x}}_i, \tilde{\boldsymbol{x}}_{i'}), \tag{14}$$

and kernel κ chosen from RBF $\kappa(\boldsymbol{x}, \boldsymbol{x}') = \exp(-\gamma\|\boldsymbol{x} - \boldsymbol{x}'\|^2)$ or Laplacian $\exp(-\gamma\|\boldsymbol{x} - \boldsymbol{x}'\|_1)$. Hyperparameters: λ (= `alpha`), γ, kernel type.

(c) Gaussian process regression (GPR). Place a GP prior $f \sim \mathcal{GP}(0, \kappa_\vartheta)$ and Gaussian likelihood $c = f(\tilde{\boldsymbol{x}}) + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, \sigma_n^2)$. Kernel hyperparameters ϑ (signal variance, length scale, noise) are fit by maximizing the log marginal likelihood

$$\vartheta^\star = \arg\max_{\vartheta} \; -\tfrac{1}{2}\boldsymbol{c}^\top (\boldsymbol{K}_\vartheta + \sigma_n^2 \boldsymbol{I})^{-1}\boldsymbol{c} - \tfrac{1}{2}\log\left|\boldsymbol{K}_\vartheta + \sigma_n^2 \boldsymbol{I}\right| - \tfrac{n}{2}\log 2\pi. \tag{15}$$

Prediction at a test $\boldsymbol{x}_*$ is Gaussian with the standard posterior mean/variance (Eqs. (25)–(26) below, applied to the $\boldsymbol{c}$ targets). Hyperparameters exposed to search: `length_scale`, `noise_level`, `alpha` (jitter).

(d) Multilayer perceptron (MLP). A feed-forward network f_θ (at most two hidden layers, width ≤ 256) minimizes penalized mean-squared error

$$\theta^\star = \arg\min_{\theta} \frac{1}{n}\sum_{i=1}^{n} \left\|\boldsymbol{c}_i - f_\theta(\tilde{\boldsymbol{x}}_i)\right\|_2^2 + \alpha \left\|\theta\right\|_2^2, \tag{16}$$

optimized by Adam. Hyperparameters: depth, width, ℓ_2 penalty α, initial learning rate. Deep operator/PINN/FNO families are deliberately out of scope for this proof of concept.

4.3 Model selection: cross-validated hyperparameter search

For a model family M and hyperparameters $\boldsymbol{\eta}$, quality is the K-fold cross-validated error measured in *original* ψ units (PCA is refit on each training fold to avoid leakage, then inverse-transformed before scoring):

$$J(M, \boldsymbol{\eta}) = \frac{1}{K}\sum_{f=1}^{K} \mathrm{RMSE}\Big(\{\boldsymbol{\psi}_i\}_{i\in V_f},\, \{\boldsymbol{\mu}_\psi^{(f)} + \boldsymbol{W}^{(f)} h_{M,\boldsymbol{\eta}}^{(f)}(\boldsymbol{x}_i)\}_{i\in V_f}\Big), \tag{17}$$

where V_f is validation fold f and the superscript (f) denotes quantities fit on the complementary training rows. The RMSE over valid grid cells Ω is

$$\mathrm{RMSE}(\boldsymbol{\psi}, \hat{\boldsymbol{\psi}}) = \sqrt{\frac{1}{|\Omega|}\sum_{(R,Z)\in\Omega} \big(\psi(R,Z) - \hat{\psi}(R,Z)\big)^2}. \tag{18}$$

The search minimizes J over the joint (family, hyperparameter) space using the Tree-structured Parzen Estimator (TPE), a sequential model-based optimizer that models $p(\boldsymbol{\eta} \mid J < J^*)$ and $p(\boldsymbol{\eta} \mid J \geq J^*)$ and samples $\boldsymbol{\eta}$ maximizing their ratio:

$$(M^\star, \boldsymbol{\eta}^\star) = \arg\min_{M,\boldsymbol{\eta}} J(M, \boldsymbol{\eta}). \tag{19}$$

Writing $\hat{h} = h_{M^\star,\boldsymbol{\eta}^\star}$ for the winner, the winning configuration is refit on *all* non-test rows to produce the deployed payload $\{\boldsymbol{W}, \boldsymbol{\mu}_\psi, \hat{h}\}$.

4.4 Reference metric: skill over the trivial baseline

To make errors interpretable and scale-free we compare against the trivial predictor that ignores $\boldsymbol{x}$ and returns the per-pixel training mean $\boldsymbol{\mu}_\psi$. Its error on an evaluation set is

$$\mathrm{RMSE}_{\mathrm{base}} = \mathrm{RMSE}\big(\{\boldsymbol{\psi}_i\}_{i\in\mathrm{eval}},\, \{\boldsymbol{\mu}_\psi\}\big), \tag{20}$$

and we define *accuracy* as the fractional error reduction

$$\text{Acc} \;=\; 100\left(1 - \frac{\text{RMSE}}{\text{RMSE}_{\text{base}}}\right)\%. \tag{21}$$

$\text{Acc} = 0$ means no better than the mean; $\text{Acc} \to 100$ means near-perfect. This quantity drives the meta-loop stopping rule (Section 6.2).

5 Phase 3: Residual-Driven Active Sampling

Given a trained surrogate, active sampling asks: *which new geometries, once solved, most improve the surrogate per solve spent?* Our answer is to learn a probabilistic model of the surrogate's own error over geometry space and sample where that error is large *or* uncertain.

5.1 Out-of-fold residuals: an honest error signal

Let $\hat{h}$ be the current winner. Its residual must be measured out-of-sample, because interpolating models (GPR, KRR) fit their own training points to ≈ 0 error. We therefore compute *out-of-fold* (OOF) residuals: partition the training pool into K folds, refit the winner's architecture on each complement, and score each sample on the fold that held it out. For sample i in fold $f(i)$,

$$r_i \;=\; \frac{1}{|\Omega_i|}\sum_{(R,Z)\in\Omega_i} \big|\,\psi_i(R,Z) - \hat{\psi}_i^{(-f(i))}(R,Z)\,\big|, \tag{22}$$

the mean absolute flux error of the winner *refit without* the fold containing i. This r_i is the empirical "how wrong is the surrogate at $\boldsymbol{x}_i$" signal.

5.2 A Gaussian process over the error surface

We now learn the function $g : \mathcal{X} \to \mathbb{R}$ mapping geometry to (log) expected surrogate error, treating $\{(\tilde{\boldsymbol{x}}_i, \log r_i)\}$ as noisy observations. We place a GP prior

$$g \sim \mathcal{GP}\big(0,\ \kappa_\vartheta\big), \qquad \log r_i = g(\tilde{\boldsymbol{x}}_i) + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma_n^2), \tag{23}$$

with an *automatic relevance determination* (ARD) squared-exponential kernel plus a white-noise term,

$$\kappa_\vartheta(\boldsymbol{x}, \boldsymbol{x}') = \sigma_f^2 \exp\left(-\tfrac{1}{2}\sum_{d=1}^{5}\frac{(x_d - x'_d)^2}{\ell_d^2}\right) + \sigma_n^2\,\delta_{\boldsymbol{x}\boldsymbol{x}'}. \tag{24}$$

Per-dimension length scales ℓ_d let the model discover which geometry axes drive error. The log target is used because error is multiplicative and heavy-tailed; the log keeps a few catastrophic samples from dominating.

The GP is a distribution over functions. Conditioning the prior on the OOF data $\boldsymbol{y} = (\log r_1, \ldots, \log r_m)^\top$ at inputs $\tilde{X} = \{\tilde{\boldsymbol{x}}_i\}$ yields a *posterior distribution over the whole space of error-surfaces g*. It is characterized by the property that, at any finite set of query geometries, the function values are jointly Gaussian. At a single candidate $\boldsymbol{x}_*$,

$$\mu_g(\boldsymbol{x}_*) = \boldsymbol{k}_*^\top(\boldsymbol{K} + \sigma_n^2\boldsymbol{I})^{-1}\boldsymbol{y}, \tag{25}$$

$$\sigma_g^2(\boldsymbol{x}_*) = \kappa_\vartheta(\boldsymbol{x}_*, \boldsymbol{x}_*) - \boldsymbol{k}_*^\top(\boldsymbol{K} + \sigma_n^2\boldsymbol{I})^{-1}\boldsymbol{k}_*, \tag{26}$$

where $K_{ii'} = \kappa_\vartheta(\tilde{\boldsymbol{x}}_i, \tilde{\boldsymbol{x}}_{i'})$ and $[\boldsymbol{k}_*]_i = \kappa_\vartheta(\boldsymbol{x}_*, \tilde{\boldsymbol{x}}_i)$. Thus $\mu_g(\boldsymbol{x}_*)$ is the posterior mean prediction of the surrogate's (log) error at $\boldsymbol{x}_*$, and $\sigma_g(\boldsymbol{x}_*)$ its posterior standard deviation — large where many plausible error-surfaces are consistent with the data but disagree, i.e. where we have not sampled. Hyperparameters $\vartheta = (\sigma_f, \{\ell_d\}, \sigma_n)$ are fit by maximizing the marginal likelihood (15) on the OOF targets.

5.3 Feasibility model

Solves fail in parts of $\mathcal{X}$, and a failed solve costs as much as a success. We learn the probability of success with a GP *classifier*: a latent GP $\phi \sim \mathcal{GP}(0, \kappa)$ mapped through a sigmoid link,

$$p(\boldsymbol{x}) \;=\; \mathbb{P}(y^{\text{feas}} = 1 \mid \boldsymbol{x}) \;=\; \sigma\big(\phi(\boldsymbol{x})\big), \qquad \sigma(z) = \frac{1}{1+e^{-z}}, \tag{27}$$

fit on *all* attempts (successes and failures) by Laplace approximation of the posterior (the non-Gaussian likelihood has no closed form). We convert the probability into an acquisition weight with an exploration floor $\rho \in (0, 1]$,

$$w(\boldsymbol{x}) = \max\big(p(\boldsymbol{x})^2,\ \rho\big). \tag{28}$$

Squaring makes a known-bad region unable to out-bid a merely well-covered feasible one on prior variance alone; the floor (default $\rho = 0.02$) ensures no region is written off permanently. Setting $\rho = 1$ disables feasibility weighting entirely.

5.4 Acquisition: upper confidence bound

We score candidate geometries by an upper-confidence-bound (UCB) rule that trades exploitation of known-high error against exploration of error-model uncertainty, modulated by feasibility. Working in log space (so the feasibility weight enters additively),

$$\boxed{a(\boldsymbol{x}) \;=\; \underbrace{\mu_g(\boldsymbol{x})}_{\text{expected error}} \;+\; \beta\, \underbrace{\sigma_g(\boldsymbol{x})}_{\text{error-model uncertainty}} \;+\; \underbrace{\log w(\boldsymbol{x})}_{\text{feasibility}}.} \tag{29}$$

The parameter $\beta \geq 0$ (clamped to $[0, 3]$, chosen by the meta-agent) controls exploration: $\beta = 0$ samples purely where the model is *predicted* worst; large β samples where the error model is *uncertain*, which is precisely how the batch reaches unvisited regions of a target envelope wider than the seed box. Equation (29) is the GP-UCB acquisition function of Bayesian optimization, applied here to the surrogate's *error* rather than to ψ itself.

5.5 Batch selection by exact variance conditioning

We must pick a *batch* of B points, not one, and they should not pile onto a single peak. We select greedily from a Sobol candidate pool $\mathcal{C}$ of size N_c (default 4096). A key fact: the GP posterior variance (26) depends only on input *locations*, not on observed values. Hence we can condition on a chosen point using a hallucinated observation at its own posterior mean (the "kriging believer" heuristic): the posterior mean is unchanged, and only the variance σ_g^2 shrinks near the chosen point. After each pick we update the Cholesky factor of $(\boldsymbol{K} + \sigma_n^2 \boldsymbol{I})$ by a rank-one border update and re-score the pool. Algorithm 1 summarizes the procedure.

The returned batch is solved by the oracle, merged into the pool, and the winner is refit — yielding immediate credit (or none) on the frozen evaluation set. When no winner yet exists (cold

Algorithm 1 Residual-driven UCB batch acquisition

Require: winner $\hat{h}$; pool $\mathcal{D}$; bounds $[\boldsymbol{\ell}, \boldsymbol{u}]$; batch B; exploration β

compute OOF residuals r_i via Eq. (22); set targets $y_i = \log r_i$
fit error-model GP g (Eq. (24)) and feasibility model p; hyperparameters by Eq. (15)
draw Sobol candidate pool $\mathcal{C} = \{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(N_c)}\} \subset [\boldsymbol{\ell}, \boldsymbol{u}]$
precompute $\mu_g(\boldsymbol{x})$, $\log w(\boldsymbol{x})$ for all $\boldsymbol{x} \in \mathcal{C}$
$\mathcal{A} \leftarrow \varnothing$
for $b = 1, \ldots, B$ **do**
    $\boldsymbol{x}^\star \leftarrow \arg\max_{\boldsymbol{x} \in \mathcal{C} \setminus \mathcal{A}} \; \mu_g(\boldsymbol{x}) + \beta\, \sigma_g(\boldsymbol{x}) + \log w(\boldsymbol{x})$ ▷ Eq. (29)
    $\mathcal{A} \leftarrow \mathcal{A} \cup \{\boldsymbol{x}^\star\}$
    condition g on $\boldsymbol{x}^\star$ (kriging believer): rank-one Cholesky update, shrinking σ_g^2 near $\boldsymbol{x}^\star$
end for
return batch $\mathcal{A}$ to solve with the oracle

start) or data is too sparse, the strategy degrades gracefully: to a model-agnostic PCA-variance acquisition (a GP over ψ coefficients, chasing predictive variance of the field itself), and finally to feasibility-weighted maximin space-filling.

5.6 Strategy selection

The meta-agent may override the automatic ladder with one of four strategies: `auto` (residual-driven if a winner exists, else uncertainty); `residual_ucb` (force Eq. (29)); `uncertainty` (model-agnostic field variance, for when the winner's residuals are still noisy); and `space_filling` (pure maximin, to blanket a newly widened region before targeting within it).

6 The Meta-Loop: Autonomous Orchestration

An outer loop drives Phases 1–3 as a sequential decision process. At each iteration t the meta-agent observes a state s_t (diagnostics of the current model: learning-curve slope, cross-seed variance, PCA spectrum, edge-hit flags, per-input residual correlations) and selects an action

$$a_t \in \{\texttt{regen\_dataset},\ \texttt{enrich\_active},\ \texttt{extend\_search},\ \texttt{terminate}\}, \tag{30}$$

respectively: append a blind space-filling batch (control arm); append a residual-driven active batch (Section 5); run another Phase-2 hyperparameter search; or stop. The action is produced by an LLM policy π_Θ expressed as a structured program that emits a typed decision; a deterministic dispatcher executes it. The policy *prompt* Θ is itself optimizable: given a dataset of (state, decision, downstream-score) traces, a reflective prompt-evolution procedure (GEPA) searches

$$\Theta^\star = \arg\max_{\Theta} \; \mathbb{E}_{\tau \sim \pi_\Theta}\big[\,\mathrm{Score}(\tau)\,\big], \tag{31}$$

where Score is a composite of hard gates (deliverables present and loadable, report parseable) and weighted quality terms (final skill over baseline, monotone improvement across iterations, budget efficiency, non-wasted iterations, agent-chosen termination). Only the decision policy is learned; the acquisition mathematics of Section 5 is fixed and deterministic for reproducibility.

6.1 Evaluation over a target envelope

Active sampling expands into geometries the seed box never covered, so the evaluation set must represent the full *target envelope* $\mathcal{E} \supseteq \mathcal{B}$, not the seed distribution. We draw one independent LHS eval set over $\mathcal{E}$, solve it once, and *freeze* it; every RMSE the loop reports is measured on these fixed samples. To reveal *where* the surrogate is weak we stratify $\mathcal{E}$ into a grid of geometry cells $\{\mathcal{E}_q\}$ (b bins per axis) and report per-cell error

$$\mathrm{RMSE}_q = \mathrm{RMSE}\big(\{\boldsymbol{\psi}_i\}_{i:\boldsymbol{x}_i \in \mathcal{E}_q},\ \{\mathcal{S}(\boldsymbol{x}_i)\}\big), \tag{32}$$

with headline summaries $\max_q \mathrm{RMSE}_q$ (worst cell) and mean-over-cells. Aggregate RMSE alone cannot distinguish "uniformly mediocre" from "excellent in the seed box, broken at high κ" — exactly the distinction active sampling exists to close.

6.2 Performance stopping criteria

The loop runs at most $T_{\max}$ iterations but stops early once the model is good enough. Three aggregate targets — an absolute RMSE τ_{abs}, a ratio ρ_r of baseline, and an accuracy A (Eq. (21)) — are the same quantity in different units and collapse to the strictest threshold

$$\tau^\star = \min\Big\{\tau_{\mathrm{abs}},\ \rho_r\,\mathrm{RMSE}_{\mathrm{base}},\ \big(1 - \tfrac{A}{100}\big)\mathrm{RMSE}_{\mathrm{base}}\Big\}, \tag{33}$$

and the loop halts at the first iteration with $\mathrm{RMSE} \le \tau^\star$. A stricter, per-region criterion halts only when *every* occupied eval cell clears an accuracy bar against *its own* baseline,

$$\min_q\ 100\left(1 - \frac{\mathrm{RMSE}_q}{\mathrm{RMSE}_{\mathrm{base},q}}\right)\ \ge\ A_{\mathrm{cell}}, \tag{34}$$

using per-cell baselines so a large-ψ region is not penalized for its scale. This "good everywhere" rule is the natural target for active learning: it cannot be satisfied by a model that is accurate only in the seed box.

7 The Agentic Code-Generation Layer (URSA)

Beyond the fixed decision policy, an agentic layer (URSA) authors open-ended, code-shaped pipeline components via a plan–execute–feedback loop: a planner LLM emits a sequence of steps from a natural-language task; an executor LLM writes and runs code for each; and, after execution, the planner re-plans given the execution history for a bounded number of feedback rounds. This layer is used where the task is genuinely open (rather than a small typed decision), most notably *representation search* — exploring alternative input featurizations and output transforms for the surrogate under a fixed evaluation protocol — and, optionally, as a code-generation backend for Phase-2 search. It is the mechanism by which the platform can extend its own data-engineering surface rather than only tuning within a fixed one.

8 Summary

Autokamak composes four mathematical layers. A finite-element solver provides ground truth for the Grad–Shafranov equation (3). A surrogate factored as PCA (10) plus a cross-validated regressor (17) emulates the flux field. A Gaussian process over the surrogate's out-of-fold residuals (22)–(26) turns "where is the model wrong" into a probabilistic surface, and a UCB rule (29)

converts it into the next equilibria to solve. An LLM meta-policy sequences data enrichment and re-tuning against explicit accuracy targets (21). The through-line is a single principle: because the oracle is expensive and the surrogate is cheap, every design choice — sampling, reduction, model selection, and above all acquisition — is aimed at extracting the most model improvement per oracle call.